

لغة تعريف البيانات (Data Definition Language – DDL)

إنشاء قاعدة بيانات جديدة (CREATE DATABASE)

الخطوة التأسيسية الأولى والركيزة الأساسية لبناء أي نظام معلوماتي أو تطبيق برمجي. لتهيئة بيئة تخزينية مستقلة ومنظمة داخل خادم قواعد البيانات (Database Server).

خطوات كتابة أمر إنشاء قاعدة بيانات جديدة: (CREATE DATABASE)

1. كتابة الأداة الأساسية للإنشاء: تبدأ الجملة بكتابة العبارة `CREATE DATABASE` وهي الكلمات المحجوزة (Keywords) في لغة SQL للإعلان عن بدء عملية تعريف وبناء كائن هيكلي جديد داخل السيرفر.

CREATE DATABASE

2. تضمين شرط التحقق الوقائي (اختياري ومعيارى): يتم كتابة العبارة `IF NOT EXISTS` مباشرة بعد أداة الإنشاء، وهي ممارسة برمجية متقدمة تضمن عدم توقف السيرفر أو حدوث خطأ نظام في حال كان اسم قاعدة البيانات مستخدماً مسبقاً على السيرفر.

IF NOT EXISTS

3. تحديد مسمى قاعدة البيانات: يتم كتابة الاسم المختار للمستودع الرقمي مثل `university_db` مع مراعاة القواعد الرقمية للتسمية بحيث يخلو من المسافات أو الرموز الخاصة عدا الشرطة السفلية. (_)

university_db

4. تحديد ترميز اللغة (اختياري ومعيارى): يتم إلحاق الأمر بعبارة ضبط الترميز (مثل `CHARACTER SET utf8mb4`) لضمان دعم قاعدة البيانات لقراءة وتخزين الحروف باللغة العربية بشكل صحيح ودون تشويه.

CHARACTER SET utf8mb4

5. إغلاق النطاق البرمجي للأمر :يتم إنهاء الجملة بوضع الفاصلة المنقوطة ;للاشارة إلى نهاية مسار الأمر الحالي، وإعلام مفسر SQL بجاهزية السكريبت للتنفيذ على السيرفر.

; ←

الشكل النهائي للكود :

```
CREATE DATABASE IF NOT EXISTS university_db CHARACTER SET utf8mb4 ;
```

اسم قاعدة البيانات **create database if not exists**

اسم نظام الترميز **character set**

آلية المقارنة والترتيب **collate**;

❖ مثال توضيحي:

```
create database if not exists database_1
```

```
character set utf8mb4
```

```
collate utf8mb4_unicode_ci;
```

إنشاء جدول جديد (CREATE TABLE)

بناء الجداول وتحديد الأعمدة والأنواع والقيود التي تحكم البيانات المدخلة لاحقاً. وتعتمد آلية عمل هذه البنية على تمرير أمر الإنشاء إلى محرك قواعد البيانات؛ ففي حال تم إسناد اسم فريد للجدول وتحديد حقل واحد على الأقل بصياغة سليمة، ينتقل المحرك مباشرة إلى تشييد الهيكل التخزيني بنجاح. أما في حالة تكرار اسم الجدول أو وجود خطأ تركيبى، فإن المحرك البرمجي يتجاوز العملية ويصدر تنبيهاً فورياً، وهو ما يضمن سلامة قاعدة البيانات ومنع تداخل الكائنات الهيكلية.

خطوات كتابة أمر إنشاء جدول جديد: (CREATE TABLE)

1. كتابة الأداة الأساسية وشرط الأمان :تبدأ الجملة بعبارَة CREATE TABLE IF NOT EXISTS وهي الكلمة المحجوزة للإعلان عن بدء تشييد هيكل جدولي جديد بشرط عدم تكرار الاسم لحماية بنية القواعد.

CREATE TABLE IF NOT EXISTS

2. تسمية الجدول وفتح النطاق :يتم كتابة الاسم المختار للجدول (مثل students)مباشرة بعد أداة الإنشاء، متبوعاً بقوس هلالِي مفتوح (لبدء تحديد عناصر الجدول.

students (

3. تعريف الأعمدة والأنواع :يتم كتابة أسماء الحقول مثل student_id وبجانب كل حقل نوع البيانات الخاص به مثل INT أو VARCHAR ، متبوعة بفاصلة ,، للانتقال للعمود التالي.

student_id INT,

student_name VARCHAR(100)

4. إغلاق نطاق الأعمدة :يتم وضع قوس هلالِي مغلق (بعد الانتهاء تماماً من تعريف آخر عمود في الجدول للإشارة إلى اكتمال مواصفات الهيكل.

)

5. إغلاق جملة الاستعلام :يتم إنهاء الأمر بالكامل بوضع الفاصلة المنقوطة ;، للإشارة إلى نهاية مسار الأمر الهيكلي الحالي وجاهزيته للتنفيذ على السيرفر.

;

الشكل النهائي للكود :

CREATE TABLE IF NOT EXISTS students(

student_id INT,

student_name VARCHAR(100)

);

أنواع البيانات في القواعد (SQL Data Types)

تُعد أنواع البيانات الركيزة الأساسية لتعريف طبيعة الحقول وتحديد المساحة التخزينية لكل عمود داخل الجدول، حيث يتمثل دورها الوظيفي في وضع قيود على نوعية المدخلات التي يُسمح للمستخدم بكتابتها. وتعتمد آلية عمل هذه البنية على قيام محرك القواعد بفحص كل قيمة يتم إدخالها؛ ففي حال كانت متوافقة مع نوع البيانات المحدد للعمود (كإدخال نص في حقل نصي)، ينتقل المحرك مباشرة إلى قبولها وتخزينها بنجاح. أما في حالة عدم التطابق (كإدخال نصوص في حقل مخصص للأرقام فقط)، فإن المحرك البرمجي يتجاوز عملية الحفظ ويصدر خطأ فورياً، وهو ما يضمن سلامة البيانات ومنع حدوث أي تضارب أو تلف في ذاكرة السيرفر.

نوع البيانات (الكلمة المحجوزة)	الاستخدام الوظيفي	السعة والحدود المسموحة	مثال تطبيقي
INT	مخصص لتخزين الأرقام الصحيحة فقط (بدون كسور) مثل المعرفات، الأعمار، والدرجات.	يخزن الأرقام بالسالب والموجب. (حتى أكثر من 2 مليار).	student_id INT
VARCHAR(الطول)	مخصص لتخزين النصوص والأسماء المتغيرة مثل أسماء الطلاب، الإيميلات، وكلمات المرور.	يجب تحديد الحد الأقصى للحروف بين قوسين (من 1 إلى 65,535 حرف).	student_name VARCHAR(100)
DATE	مخصص لتخزين التواريخ الزمنية فقط دون الوقت مثل تاريخ الميلاد أو تاريخ التسجيل.	يدعم التواريخ بصيغة ثابتة وموحدة هي YYYY-MM-DD (سنة-شهر-يوم).	birth_date DATE

student_bio TEXT	مخصص لتخزين النصوص الطويلة جداً والمفتوحة مثل المقالات، الوصف، أو حلول الأسئلة المقالية. طول مسبق.	مساحة مفتوحة تتسع حتى 65,535 حرف ولا تتطلب تحديد	TEXT
------------------	--	--	------

نوع البيانات اسم العمود الأول
نوع البيانات اسم العمود الثاني

اسم الجدول **create table if not exists**

(
القيود نوع البيانات اسم العمود الأول
القيود نوع البيانات اسم العمود الثاني
);

❖ مثال توضيحي:

```
create table if not exists students (  
    student_id int primary key auto_increment,  
    student_name varchar(100) not null
```

التعديل في بنية جدول (ALTER TABLE)

تُعد هذه المهارة الأداة الأساسية لتحديث وتطوير الهيكل التخزيني بعد إنشائه، حيث يتمثل دورها الوظيفي في إجراء تعديلات على أعمدة الجدول (مثل الإضافة، أو الحذف، أو تغيير أنواع البيانات) دون الحاجة لإسقاط الجدول أو خسارة البيانات المخزنة بداخله. وتعتمد آلية عمل هذه البنية على تمرير أمر التعديل إلى محرك قواعد البيانات؛ ففي حال تم تحديد اسم جدول موجود بالفعل وصياغة التعديل الفرعي بشكل سليم، ينتقل المحرك مباشرة إلى إعادة هيكلة الجدول في الذاكرة بنجاح. أما في حالة عدم وجود الجدول في القاعدة أو كتابة أمر فرعي خاطئ، فإن المحرك البرمجي يتجاوز العملية ويصدر تنبيهاً فورياً، وهو ما يضمن استقرار البنية الهيكلية للنظام.

خطوات كتابة أمر التعديل العام: (ALTER TABLE)

1. كتابة الأداة الأساسية: تبدأ الجملة بعبارة ALTER TABLE وهي الكلمة المحجوزة في لغة الاستعلام للإعلان عن بدء عملية تعديل أو إعادة هيكلة لجدول قائم بالفعل.

ALTER TABLE

2. تحديد الجدول المستهدف: يتم كتابة الاسم الفعلي للجدول المراد تعديل بنيته (مثل students) مباشرة بعد أداة التعديل العامة.

students

3. تحديد الإجراء الفرعي: يتم كتابة أمر التعديل الخاص (مثل ADD لإضافة عمود، أو DROP لحذف عمود) لتوجيه المحرك لنوع التغيير المطلوب.

ADD

4. تضمين مواصفات الأعمدة: كتابة أسماء الحقول المتأثرة بالتعديل متنوعة بأنواع بياناتها والقيود الخاصة بها بناءً على الإجراء الفرعي المختار.

5. إغلاق جملة الاستعلام: يتم إنهاء الأمر بالكامل بوضع الفاصلة المنقوطة ؛ للإشارة إلى نهاية مسار أمر التعديل الهيكلي وجاهزيته للتنفيذ الفوري على السيرفر.

الشكل النهائي للكود :

ALTER TABLE students **ADD** student_phone **VARCHAR(15)** **NOT NULL** ;

ALTER TABLE مواصفات العمود / نوع الإجراء الفرعي / اسم الجدول

ALTER TABLE students

MODIFY student_name **VARCHAR(150)** **NOT NULL**;

إضافة حقل جديد إلى الجدول (ALTER ADD)

تُعد هذه المهارة التطبيقية الأداة المباشرة لتوسيع بنيان الجداول القائمة بالفعل، حيث يتمثل دورها الوظيفي في إدراج أعمدة جديدة لم تكن موجودة أثناء عملية الإنشاء الأولى، وذلك لاستيعاب متغيرات أو بيانات مستجدة في النظام دون الحاجة لحذف الجدول أو المساس بالبيانات القديمة المحفوظة بداخل الحقول الأخرى. وتعتمد آلية عمل هذه البنية على قيام محرك القواعد باستقبال أمر الإضافة؛ ففي حال كان اسم الحقل الجديد فريداً ولم يتم تكراره داخل نفس الجدول، ينتقل المحرك مباشرة إلى حجز مساحة تخزينية له وتثبيته بنجاح. أما في حالة وجود حقل بنفس الاسم مسبقاً أو وجود خطأ في نوع البيانات المحددة، فإن المحرك البرمجي يتجاوز العملية ويصدر تنبيهاً فورياً، وهو ما يحمي بنية الجدول وجداوله من التشوه أو التكرار غير المنطقي.

خطوات كتابة أمر إضافة حقل جديد: (ALTER ADD)

1. استدعاء أداة التعديل العامة: تبدأ الجملة البرمجية بكتابة العبارة المحجوزة **ALTER TABLE** للإعلان عن بدء تعديل كائن جدولي قائم.

ALTER TABLE

2. تحديد الجدول المستهدف: ترك مسافة فارغة واحدة، ثم كتابة الاسم الفعلي للجدول المراد توسيع بنيته مثل students .

```
ALTER TABLE students
```

3. تضمين أمر الإضافة الفرعي: الانتقال لسطر جديد وكتابة الكلمة المحجوزة ADD لتوجيه المحرك بأن الإجراء الحالي هو إدراج عمود جديد.

```
1 ALTER TABLE students
2 ADD
```

4. تسمية وتوصيف الحقل المستجد: كتابة الاسم المختار للعمود الجديد مثل student_phone متبوعاً مباشرة بنوع بياناته مثل VARCHAR(15) والقيود الحاكمة له.

```
1 ALTER TABLE students
2 ADD student_phone VARCHAR(15) NOT NULL ;
```

5. إغلاق جملة الاستعلام: إنهاء السكريبت بوضع الفاصلة المنقوطة ؛ للإشارة إلى نهاية مسار الأمر الهيكلي الحالي وجاهزيته للتنفيذ الفوري على السيرفر.

```
1 ALTER TABLE students
2 ADD student_phone VARCHAR(15) NOT NULL ;
```


ALTER TABLE اسم الجدول

ADD القيود / نوع البيانات / اسم العمود الجديد

```
ALTER TABLE students
```

```
ADD student_phone VARCHAR(15) NOT NULL;
```

تحديد موقع العمود الجديد عند الإضافة (ALTER ... AFTER)

تُعد هذه المهارة التطبيقية للتحكم في الترتيب البصري والمنطقي لأعمدة الجدول، حيث يتمثل دورها في إدراج الحقل المستجد في موقع محدد وعقب عمود معين تم إنشاؤه مسبقاً، بدلاً من ترك المحرك يضعه تلقائياً في نهاية الجدول. وتكمن الأهمية لهذه المهارة في الحفاظ على ترابط البيانات المتشابهة مجاورة لبعضها البعض (مثل وضع حقل الهاتف مباشرة بعد حقل الاسم)، وتعتمد آلية عمل هذه البنية على تمرير الكلمة المفتاحية **AFTER** متبوعة بالعمود المراد اللحاق به؛ ففي حال كان العمود المرجعي موجوداً داخل الجدول، ينتقل المحرك مباشرة إلى تعديل ترتيب الأعمدة. أما في حالة عدم وجود العمود المرجعي المستهدف أو حدوث خطأ في الصياغة، فإن المحرك البرمجي يتجاوز العملية ويصدر تنبيهاً فورياً، مما يحمي الهيكل التنظيمي للجدول من العشوائية.

خطوات كتابة أمر تحديد موقع العمود الجديد: (ALTER ... AFTER)

1. استدعاء أداة التعديل العامة: تبدأ الجملة البرمجية بكتابة العبارة المحجوزة **ALTER TABLE** للإعلان عن بدء تعديل كائن جدولي قائم.

```
1 ALTER TABLE
```

2. تحديد الجدول المستهدف: تترك مسافة فارغة واحدة، ثم كتابة الاسم الفعلي للجدول المراد تعديل ترتيبه مثل **students**.

```
1 ALTER TABLE students
```

4. تضمين أمر الإضافة الموضوعي: الانتقال لسطر جديد وكتابة الكلمة المحجوزة ADD متبوعة باسم العمود الجديد ونوع بياناته مثل student_email VARCHAR(100).

```
1 ALTER TABLE students
2 ADD student_email VARCHAR(100)
```

5. تحديد الموقع والعمود المرجعي: كتابة الكلمة المفتاحية AFTER لتعريف المحرك بالموقع، يليه مباشرة الاسم الدقيق للعمود المراد اللحاق به مثل student_name.

```
1 ALTER TABLE students
2 ADD student_email VARCHAR(100) AFTER student_name ;
```

6. إغلاق جملة الاستعلام: إنهاء السكريبت بوضع الفاصلة المنقوطة ؛ للإشارة إلى نهاية مسار الأمر الهيكلي الحالي وجاهزيته للتنفيذ الفوري على السيرفر .

```
1 ALTER TABLE students
2 ADD student_email VARCHAR(100) AFTER student_name ;
```

ALTER TABLE اسم الجدول

ADD اسم العمود المرجعي AFTER نوع البيانات / نوع اسم العمود الجديد

```
ALTER TABLE students
ADD student_email VARCHAR(100) AFTER student_name;
```

إسقاط قاعدة بيانات كاملة (DROP DATABASE)

تُعد هذه المهارة الأداة الإدارية القصوى لإزالة وتدمير البيئة التخزينية للمشروع بالكامل، حيث يتمثل دورها الوظيفي في التخلص من قاعدة البيانات المركزية ومحو كل ما تحتويه من جداول، وسجلات، وعلاقات، وقيود من جذورها. وتعتمد آلية عمل هذه البنية على توجيه أمر مباشر إلى خادم قواعد البيانات؛ ليقوم بمسح المجلدات والملفات الفيزيائية الخاصة بتلك القاعدة نهائياً من القرص الصلب للسيرفر، وهو ما يوفر مساحة

الخادم الفورية. وتتطلب هذه المهارة حذراً شديداً من الطالب، لأن تنفيذها يعني فقدان كافة بيانات النظام التعليمي بشكل دائم ولا يمكن التراجع عنه أو استعادته.

خطوات كتابة أمر إسقاط قاعدة البيانات: (DROP DATABASE)

1. استدعاء أداة الإسقاط الكلية: تبدأ الجملة البرمجية بكتابة الكلمة المحجوزة DROP بأحرف كبيرة للإعلان عن بدء عملية حذف هيكلي نهائي.
2. تحديد الكائن المركزي: ترك مسافة فارغة واحدة، ثم كتابة الكلمة المفتاحية DATABASE لتوجيه المحرك بأن الحذف يستهدف قاعدة بيانات بالكامل وليس جدولاً فرعياً.
3. تضمين شرط الأمان الوقائي: كتابة العبارة المعيارية IF EXISTS لمباشرة لحماية النظام من التوقف أو إظهار أخطاء في حال كانت القاعدة غير موجودة مسبقاً.
4. تحديد مسمى قاعدة البيانات: كتابة الاسم الفعلي والدقيق لقاعدة البيانات المستهدفة بالتدمير والإزالة (مثل database_1).
5. إغلاق جملة الاستعلام: إنهاء السكريبت بوضع الفاصلة المنقوطة ؛ للإشارة إلى نهاية مسار الأمر الإداري وجاهزيته للتنفيذ الفوري على السيرفر.

DROP DATABASE IF EXISTS اسم قاعدة البيانات

```
DROP DATABASE IF EXISTS database_1;
```

إسقاط جدول معين من قاعدة البيانات (DROP TABLE)

تُعد هذه المهارة الأداة البرمجية المخصصة لتفكيك وإزالة هيكل جدولي محدد داخل قاعدة البيانات، حيث يتمثل دورها الوظيفي في التخلص من الجدول المستهدف ومحو أعمدته وسجلاته بالكامل لتطهير بيئة العمل، دون أن يؤثر ذلك على باقي الجداول الأخرى الموجودة في قاعدة البيانات. وتعتمد آلية عمل هذه البنية على توجيه أمر الإسقاط إلى محرك القواعد؛ ليقوم فوراً بحذف المساحة التخزينية المخصصة لهذا الجدول وإزاحته نهائياً.

من شجرة كائنات النظام. ويجب على الطالب توخي الحذر الشديد عند كتابة هذا الأمر، نظراً لأن تدمير الجدول يترتب عليه ضياع كافة البيانات المخزنة بداخله بشكل دائم ولا يمكن استرجاعها.

خطوات كتابة أمر إسقاط جدول معين: (DROP TABLE)

1. استدعاء أداة الإسقاط الكلية: تبدأ الجملة البرمجية بكتابة الكلمة المحجوزة DROP بأحرف كبيرة للإعلان عن بدء عملية حذف هيكلي نهائي.
2. تحديد كائن الهيكل الجدولي: ترك مسافة فارغة واحدة، ثم كتابة الكلمة المفتاحية TABLE لتوجيه المحرك بأن الحذف يستهدف جدولاً فرعياً محدداً.
3. تضمين شرط الأمان الوقائي: كتابة العبارة المعيارية IF EXISTS لمباشرة لحماية السكربت من التوقف أو إظهار أخطاء في حال كان الجدول محذوفاً بالفعل.
4. تحديد المسمى الدقيق للجدول: كتابة الاسم الفعلي والتنفيذي للجدول المراد تدميره نهائياً من قاعدة البيانات (مثل students).
5. إغلاق جملة الاستعلام: إنهاء السكربت بوضع الفاصلة المنقوطة ؛ للإشارة إلى نهاية مسار الأمر الإداري وجاهزيته للتنفيذ الفوري على السيرفر.

اسم الجدول المراد حذفه DROP TABLE IF EXISTS

DROP TABLE IF EXISTS students;

عرض قواعد البيانات والجدول المتاحة (SHOW)

تُعد هذه المهارة الأداة البرمجية الاستكشافية الأولى داخل خادم قواعد البيانات، حيث يتمثل دورها الوظيفي في استعراض وسرد الكائنات الهيكلية الموجودة على السيرفر (سواء كانت أسماء قواعد البيانات المتاحة، أو أسماء الجداول داخل قاعدة معينة). وتكمن الأهمية التعليمية والتنظيمية لهذه المهارة في تمكين المطور من فحص البيئة الحالية والتأكد من وجود الكائنات قبل الشروع في التعامل معها. وتعتمد آلية عمل هذه البنية على إرسال استعلام استكشافي لمحرك القواعد؛ ليقوم فوراً بقراءة جدول الفهارس المركزي في السيرفر

وعرض الأسماء المطلوبة على شكل قائمة منظمة، مما يمنع الطالب من التخمين العشوائي لأسماء الجداول أو القواعد.

خطوات كتابة أمر العرض والاستكشاف:(SHOW)

1. استدعاء أداة العرض: تبدأ الجملة البرمجية بكتابة الكلمة المحجوزة SHOW بأحرف كبيرة للإعلان عن رغبتك في استكشاف كائنات خادم البيانات.

```
1 SHOW
```

2. تحديد نوع الكائن المراد سرده: تترك مسافة فارغة واحدة، ثم كتابة صيغة الجمع للكائن المستهدف مثل كتابة DATABASES لعرض كل القواعد، أو TABLES لعرض جداول القاعدة الحالية.

```
1 SHOW DATABASES
```

3. تأكيد الصياغة الهيكلية: التأكد من خلو الأمر من أي أسماء فرعية في حال كان الغرض هو العرض الشامل لكافة القواعد المتاحة على السيرفر.

4. مراعاة حالة القاعدة النشطة: عند الرغبة في عرض الجداول (SHOW TABLES)، يجب التأكد أولاً من أن الطالب قام بفتح قاعدة البيانات واختيارها لتظهر الجداول التابعة لها.

5. إغلاق جملة الاستعلام الاستكشافية: إنهاء السطر بوضع الفاصلة المنقوطة ؛ للإشارة إلى نهاية مسار أمر العرض وجاهزيته للتنفيذ الفوري.

```
1 SHOW DATABASES ;
```

نوع الكائنات بصيغة الجمع **SHOW**

```
SHOW DATABASES;
```

تفريغ جداول البيانات مع الحفاظ على الهيكل البرمجي (TRUNCATE TABLE)

تُعد هذه المهارة الأداة البرمجية السريعة والفعالة لتطهير الجداول من كافة السجلات والبيانات المخزنة بداخلها دفعة واحدة، حيث يتمثل دورها الوظيفي في تفريغ الجدول بالكامل ليصبح فارغاً تماماً، مع الإبقاء على الهيكل البنائي للأعمدة، والأنواع، والقيود، والمؤشرات كما هي دون أدنى تغيير. وتكمن الأهمية التعليمية والعملية لتطبيق هذا الأمر في تنظيف جداول الاختبارات والتجارب المعملية لإعادة استخدامها من جديد، وتعتمد آلية عمل هذا الأمر على إسقاط البيانات فيزيائياً وإعادة تهيئة العدادات الرقمية التلقائية للجدول من البداية، مما يجعله أسرع بكثير من أوامر الحذف التقليدية، ويتطلب من الطالب دقة بالغة لأن عملية تفريغ البيانات نهائية ولا يمكن التراجع عنها.

خطوات كتابة أمر تفريغ الجدول: (TRUNCATE)

1. استدعاء أداة التفريغ الهيكلية: تبدأ الجملة البرمجية بكتابة الكلمة المحجوزة TRUNCATE بأحرف كبيرة للإعلان عن بدء عملية تفريغ ومحو شامل للسجلات.

TRUNCATE

2. تحديد كائن الهيكل الجدولي: ترك مسافة فارغة واحدة، ثم كتابة الكلمة المفتاحية TABLE لتوجيه المحرك بأن الإجراء يستهدف تفريغ محتويات جدول.

TRUNCATE TABLE

3. تحديد المسمى الدقيق للجدول: كتابة الاسم الفعلي والتنفيذي للجدول المراد مسح سجلاته بالكامل من قاعدة البيانات مثل students .

TRUNCATE TABLE students

4. التأكد من سلامة التبعية: مراعاة أن يكون الجدول مستقلاً أو لا تمنع قيود الربط الخارجية عملية تفريغه لضمان تنفيذ الأمر بنجاح على السيرفر.

5. إغلاق جملة الاستعلام الهيكلية :إنهاء السكريبت بوضع الفاصلة المنقوطة ؛للإشارة إلى نهاية مسار أمر التفريغ وجاهزيته للتنفيذ الفوري.

```
TRUNCATE TABLE students;
```

اسم الجدول المراد تفريغه **TRUNCATE TABLE**;

```
TRUNCATE TABLE students;
```

اختيار قاعدة البيانات للعمل عليها وتفعيلها (USE)

تُعد هذه المهارة الأداة التوجيهية الأساسية داخل خادم قواعد البيانات، حيث يتمثل دورها الوظيفي في تحديد وتنشيط قاعدة بيانات معينة من بين القواعد المتاحة على السيرفر، لجعله المستودع الافتراضي الحالي لكافة الأوامر اللاحقة. وتكمن الأهمية التعليمية والتنظيمية لهذه المهارة في إعلام محرك القواعد بالبيئة التي سيتم إنشاء الجداول أو استعلام البيانات بداخلها، مما يمنع حدوث تشتت أو خطأ عدم تحديد بيئة العمل الافتراضية. وتعتمد آلية عمل هذا الأمر على نقل مؤشر التنفيذ مباشرة إلى داخل حدود القاعدة المختارة، وهو ما يضمن توجيه كافة عمليات تشييد الهياكل أو البيانات المستقبلية إليها بدقة وبشكل تلقائي.

خطوات كتابة أمر اختيار قاعدة البيانات: (USE)

1. استدعاء أداة التنشيط: تبدأ الجملة البرمجية بكتابة الكلمة المحجوزة USE بأحرف كبيرة للإعلان عن رغبتك في فتح وتنشيط بيئة عمل محددة.

```
1 USE
```

2. تحديد قاعدة البيانات المستهدفة: ترك مسافة فارغة واحدة بعد الأمر، ثم كتابة الاسم الفعلي لقاعدة البيانات المراد الدخول إليها مثل database_1.

```
1 USE database_1
```

3.

4. **مطابقة الاسم**: مراعاة مطابقة الحروف والرموز الخاصة باسم القاعدة كما تم إنشاؤها مسبقاً على السيرفر لتفادي ظهور أخطاء عدم العثور على الكائن.
5. **التأكد من التأسيس المسبق**: يجب أن يراعي الطالب أن هذا الأمر لا ينشئ قاعدة جديدة، بل يتطلب وجود قاعدة بيانات حقيقية تم تشييدها مسبقاً على الخادم.
6. **إغلاق جملة الاستعلام التوجيهية**: إنهاء السطر بوضع الفاصلة المنقوطة ؛ للإشارة إلى نهاية مسار أمر الاختيار وجاهزيته للتنفيذ الفوري.

1 **USE database_1 ;**

؛ اسم قاعدة البيانات المراد تنشيطها **USE**

USE database_1;

يستعرض هيكل الجدول الداخلي وتفاصيل أعمده ونوع بياناتها باستخدام الأمر (DESCRIBE)

تُعد هذه المهارة الأداة البرمجية الاستكشافية المخصصة لفحص البنية الهيكلية للجدول من الداخل، حيث يتمثل دورها الوظيفي في استعراض تفاصيل الأعمدة، وأنواع البيانات المسندة إليها، والقيود الحاكمة لكل حقل (مثل تحديد المفتاح الرئيسي أو الحقول المسموح بتركها فارغة). وتكمن الأهمية التعليمية والعملية لتطبيق هذا الأمر في تمكين الطالب من مراجعة جودة التشييد الهيكلي للجدول والتأكد من مطابقتها للمواصفات المطلوبة قبل الشروع في عمليات إدخال البيانات، وتعتمد آلية عمله على قراءه قاموس البيانات الداخلي للسيرفر وعرض مواصفات الجدول في جدول مرئي منظم يتضمن أسماء الحقول، أنواعها، والقيود المرتبطة بها.

خطوات كتابة أمر استعراض هيكل الجدول: (DESCRIBE)

1. **استدعاء أداة الوصف الهيكلي**: تبدأ الجملة البرمجية بكتابة الكلمة المحجوزة DESCRIBE بأحرف كبيرة للإعلان عن رغبتك في استكشاف تفاصيل هيكل داخلي.

DESCRIBE

2. **تحديد الجدول المستهدف**: ترك مسافة فارغة واحدة، ثم كتابة الاسم الفعلي والتنفيذي للجدول المراد فحص أعمدته مثل students.

DESCRIBE students

3. **مراعاة دقة الحروف**: التأكد من كتابة اسم الجدول بشكل صحيح ومطابق تماماً لما هو مخزن في قاعدة البيانات النشطة لتجنب أخطاء النظام.
4. **استخدام الاختصار القياسي**: يمكن للطلاب اختصار الكلمة إلى DESC فقط، حيث تؤدي نفس الوظيفة البرمجية تماماً على السيرفر لتسهيل الكتابة.

DESC students

5. **إغلاق جملة الاستعلام الاستكشافية**: إنهاء السطر بوضع الفاصلة المنقوطة ؛ للإشارة إلى نهاية مسار أمر الوصف وجاهزيته للتنفيذ الفوري /*.

DESCRIBE students ;

DESC students ;

؛ اسم الجدول المراد استعراض تفاصيله **DESCRIBE**

DESCRIBE students;

تطبيق القيود البرمجية Constraints على أعمدة الجدول لضمان سلامة وصحة البيانات المدخلة

تُعد هذه المهارة الأداة البرمجية المخصصة لفرض قواعد صارمة ومحددات جودة على البيانات داخل الجداول، حيث يتمثل دورها الوظيفي في منع دخول أي بيانات خاطئة، أو عشوائية، أو ناقصة تخالف منطق

النظام التعليمي. وتكمن الأهمية العملية والوقائية لتطبيق القيود في حماية الجداول من التلف وضمان دقة وصحة المعلومات المخزنة بداخلها؛ وتعتمد آلية عملها على فحص المحرك للبيانات لحظة الإدخال، فإذا كانت مطابقة لل قيد سمح بحفظها، وإن خالفته أوقف المحرك العملية بالكامل وأصدر رسالة خطأ صريحة لحماية الجدول.

خطوات كتابة وتطبيق القيود البرمجية: (Constraints)

1. **تحديد موقع القيد**: يتم صياغة القيد مباشرة بعد تحديد اسم العمود ونوع بياناته، أو يتم تجميعه في نهاية أمر إنشاء الجدول.

```
1 CREATE TABLE IF NOT EXISTS courses (
```

2. **استدعاء الكلمة المفتاحية (اختياري)**: كتابة الكلمة المحجوزة CONSTRAINT متبوعة باسم مخصص للقيد لسهولة التعرف عليه وتعديله لاحقاً عند إدارة الجدول.

```
1 CREATE TABLE IF NOT EXISTS courses (  
2 |     course_id INT CONSTRAINT
```

3.

4. **تحديد نوع القيد المناسب**: كتابة القيد المطلوب بأحرف كبيرة مثل NOT NULL لمنع الفراغ، أو UNIQUE لمنع التكرار بناءً على حاجة الحقل.

```
1 CREATE TABLE IF NOT EXISTS courses (  
2 |     course_id INT CONSTRAINT pk_course PRIMARY KEY,  
3 |     course_name VARCHAR(100) CONSTRAINT uq_name UNIQUE NOT NULL
```

5. **الفصل بين القيود المتعددة**: ترك مسافة فارغة واحدة في حال رغبة الطالب في تطبيق أكثر من قيد على عمود واحد (مثل جعل الحقل فريداً وغير فارغ معاً).

```
1 CREATE TABLE IF NOT EXISTS courses (  
2 |     course_id INT CONSTRAINT pk_course PRIMARY KEY,  
3 |     course_name VARCHAR(100) CONSTRAINT uq_name UNIQUE NOT NULL  
4 |
```

6. تثبيت القيد هيكلياً: إنهاء تعريف الحقل بفاصلة ,، للانتقال للعمود التالي، أو وضع الفاصلة المنقوطة . إذا كان القيد في نهاية الأمر لتنفيذه على السيرفر .

```
1 CREATE TABLE IF NOT EXISTS courses (  
2     course_id INT CONSTRAINT pk_course PRIMARY KEY,  
3     course_name VARCHAR(100) CONSTRAINT uq_name UNIQUE NOT NULL  
4 );
```

مثال تطبيقي (بدون تسمية القيد - الطريقة المباشرة والأكثر شيوعاً):

CREATE TABLE IF NOT EXISTS اسم الجدول

```
(  
    اسم القيد البرمجي نوع البيانات اسم العمود  
    اسم القيد البرمجي نوع البيانات اسم العمود  
);
```

```
CREATE TABLE IF NOT EXISTS courses  
(  
    course_id INT NOT NULL,  
    course_name VARCHAR(100) NOT NULL  
);
```

مثال تطبيقي آخر (مع تسمية القيد ودمج قيود متعددة):

```
CREATE TABLE IF NOT EXISTS courses (  
  
    course_id INT CONSTRAINT pk_course PRIMARY KEY,  
  
    course_name VARCHAR(100) CONSTRAINT uq_name UNIQUE NOT NULL  
  
);
```

تعيين المفتاح الرئيسي Primary Key لحقل محدد لضمان فريدة السجلات وعدم تكرارها

تُعد هذه المهارة الأداة الهيكلية الأهم لتحديد الهوية الفريدة لكل سجل داخل الجدول، حيث يتمثل دورها الوظيفي في تعيين عمود محدد (أو مجموعة أعمدة) كمفتاح رئيسي يحظر تماماً تكرار قيمه أو تركها فارغة. وتكمن الأهمية العملية والتعليمية لتطبيق المفتاح الرئيسي في تمكين النظام من الوصول الفوري لأي سجل بدقة مألوفة، ومنع تداخل أو تشابه بيانات الطلاب أو المستخدمين؛ وتعتمد آلية عمله على بناء فهرس فريد وصارم من قِبل محرك قواعد البيانات، بحيث يتم رفض أي محاولة إدخال لقيمة موجودة مسبقاً داخل هذا الحقل وإصدار خطأ فوري يحمي الجدول من التكرار العشوائي.

خطوات كتابة وتعيين المفتاح الرئيسي: (Primary Key)

1. تحديد حقل الهوية: اختيار العمود المناسب ليكون معبراً عن الهوية الفريدة للسجل مثل معرف الطالب student_id .

```
CREATE TABLE IF NOT EXISTS students (
```

2. كتابة نوع البيانات: كتابة نوع البيانات الرقمي الملائم وعادة ما يكون INT متبوعاً بمسافة فارغة.

```
1 CREATE TABLE IF NOT EXISTS students (  
2     student_id INT  
-
```

3. استدعاء أداة القيد الرئيسي: كتابة العبارة المحجوزة PRIMARY KEY بأحرف كبيرة مباشرة بعد نوع البيانات لتعريف المحرك بطبيعة الحقل الصارمة.

```
1 CREATE TABLE IF NOT EXISTS students (  
2     student_id INT PRIMARY KEY AUTO_INCREMENT,
```

4. دمج خصائص التسهيل (اختياري): يمكن إلحاق القيد بخاصية الزيادة التلقائية ليتولى السيرفر توليد الأرقام الفريدة تلقائياً دون تدخل المستخدم.

```
3     student_name VARCHAR(100) NOT NULL
```

5. الفصل الهيكلي: وضع الفاصلة العادية ،في نهاية السطر للانتقال لتشديد باقي أعمدة الجدول، أو إغلاق الأمر بالفاصلة المنقوطة.

4) ;

```
CREATE TABLE IF NOT EXISTS اسم_الجدول
(
    PRIMARY KEY, اسم_عمود_المفتاح نوع_ اسم_العمود (المفتاح الأساسي)
);
```

```
CREATE TABLE IF NOT EXISTS students (
    student_id INT PRIMARY KEY,
);
```

تحديد المفتاح الأجنبي Foreign Key لربط الجداول ببعضها وتفعيل خاصية التكامل المرجعي

تُعد هذه المهارة الأداة الهندسية الأساسية لبناء العلاقات بين الجداول المختلفة داخل قاعدة البيانات العلائقية، حيث يتمثل دورها الوظيفي في ربط حقل في الجدول الحالي (يسمى الجدول التابع) بحقل المفتاح الرئيسي في جدول آخر (يسمى الجدول الأصلي). وتكمن الأهمية العملية والتعليمية للمفتاح الأجنبي في تفعيل خاصية "التكامل المرجعي"، والتي تضمن صحة العلاقات؛ بحيث يمنع محرك قواعد البيانات إدخال أي قيم عشوائية في الجدول التابع لا يكون لها أصل أو وجود حقيقي في الجدول الرئيسي (مثل منع تسجيل طالب في مقرر دراسي غير موجود مسبقاً في جدول المقررات).

خطوات كتابة وتعيين المفتاح الأجنبي: (Foreign Key)

1. تأسيس الحقل المحلي: كتابة اسم العمود المراد تخصيصه للربط داخل الجدول الحالي وتحديد نوع بياناته (ويجب أن يطابق تماماً نوع بيانات المفتاح الرئيسي في الجدول الأصلي).

```
1 CREATE TABLE IF NOT EXISTS student_courses (
2     | course_id INT,
```

2. استدعاء أداة المفتاح الأجنبي: الانتقال إلى سطر جديد في نهاية أعمدة الجدول وكتابة العبارة المحجوزة FOREIGN KEY بأحرف كبيرة .

3 FOREIGN KEY

3. تحديد العمود المحلي: وضع قوسين هلاليين () بعد الأداة وكتابة اسم الحقل المحلي المراد تحويله لمفتاح رابط بين القوسين.

3 FOREIGN KEY (course_id)

4. تحديد الجدول المستهدف: كتابة الكلمة المفتاحية REFERENCES متبوعة مباشرة باسم الجدول الأصلي المراد الارتباط به (مثل جدول الأقسام أو المقررات).

3 FOREIGN KEY (course_id) REFERENCES courses (course_id)

5. تحديد المفتاح المستهدف وإغلاق الأمر: وضع قوسين هلاليين وكتابة اسم حقل المفتاح الرئيسي للجدول الأصلي بداخلهما، ثم إغلاق جملة إنشاء الجدول بالكامل.

```
1 CREATE TABLE IF NOT EXISTS student_courses (  
2     course_id INT,  
3     FOREIGN KEY (course_id) REFERENCES courses (course_id)  
4 );
```

```
CREATE TABLE IF NOT EXISTS student_courses (  
    course_id INT,  
    FOREIGN KEY (course_id) REFERENCES courses (course_id)  
);
```

ضبط القيمة الافتراضية للعمود باستخدام القيد DEFAULT لتعويض الحقول في حال عدم الإدخال

تُعد هذه المهارة الأداة البرمجية المخصصة لتأمين حقول الجدول من البيانات المفقودة، حيث يتمثل دورها الوظيفي في إسناد قيمة تلقائية ومحددة مسبقاً للعمود في حال قيام المستخدم بحفظ السجل دون إدخال قيمة لهذا الحقل بشكل صريح. وتكمن الأهمية العملية والتعليمية لتطبيق قيد القيمة الافتراضية في ضمان استمرارية تدفق البيانات داخل النظام وتجنب وجود حقول فارغة عشوائية (مثل ضبط قيمة افتراضية لتاريخ التسجيل لتكون تاريخ اليوم الحالي، أو ضبط حالة الطالب الافتراضية لتكون "نشط")؛ وتعتمد آلية عمله على قيام محرك القواعد بفحص أمر الإدخال، فإذا وجد الحقل غائباً، قام تلقائياً بملء الفراغ بالقيمة المحجوزة مسبقاً دون إصدار أي خطأ.

خطوات كتابة وضبط القيمة الافتراضية: (DEFAULT)

1. **تحديد الحقل المستهدف**: كتابة اسم العمود المراد تطبيق الخاصية عليه وتحديد نوع بياناته المناسب (سواء رقمي أو نصي).

`Student_Status VARCHAR(20)`

2. **استدعاء أداة القيمة الافتراضية**: ترك مسافة فارغة واحدة، ثم كتابة العبارة المحجوزة DEFAULT بأحرف كبيرة مباشرة بعد نوع البيانات.

`Student_Status VARCHAR(20) DEFAULT`

3. **تحديد القيمة المعوضة**: كتابة القيمة المراد تثبيتها بعد الأداة مباشرة (تكتب الأرقام مباشرة، بينما تُحاط القيم النصية بعلامات تنصيص مفردة).

`Student_Status VARCHAR(20) DEFAULT 'Active'`

4. **تضمين الدوال الديناميكية (اختياري)**: يمكن استخدام دالة زمنية مثل CURRENT_DATE لتعويض حقول التاريخ بوقت النظام الحالي تلقائياً.

5. الفصل الهيكلي :وضع الفاصلة العادية ,في نهاية السطر للانتقال لتعريف باقي أعمدة الجدول، أو إنهاء الاستعلام بالفاصلة المنقوطة.

```
CREATE TABLE IF NOT EXISTS Students (  
    Student_Id INT PRIMARY KEY,   
    Student_Status VARCHAR(20) DEFAULT 'Active'  
);
```

CREATE TABLE IF NOT EXISTS اسم الجدول (
القيمة الافتراضية **DEFAULT** نوع البيانات اسم العمود
);

```
Create Table If Not Exists Students (  
    Student_Id Int Primary Key,  
    Student_Status Varchar(20) Default 'Active'  
);
```

صياغة شرط التحقق وقيد الفحص المخصص للبيانات الرقمية أو النصية باستخدام القيد (CHECK)

تُعد هذه المهارة الأداة البرمجية المتقدمة لفرض شروط منطقية ومخصصة على القيم المدخلة، حيث يتمثل دورها الوظيفي في التحقق من أن القيمة تقع داخل نطاق محدد أو تطابق معياراً معيناً قبل السماح بحفظها في الجدول. وتكمن الأهمية العملية والتعليمية لقيد الفحص في حماية القواعد من المدخلات غير المنطقية التي لا تتماشى مع الشروط التربوية (مثل التحقق من أن درجات الطلاب تقع بين 0 و 100، أو أن عمر الطالب أكبر من 17 عاماً)؛ وتعتمد آلية عمله على قيام محرك القواعد باختبار الشرط لحظة الإدخال، فإذا تحقق الشرط تم الحفظ بنجاح، وإذا تم تجاوزه يرفض المحرك العملية فوراً ويصدر خطأً يحمي الحقل من البيانات الزائفة.

خطوات كتابة وصياغة شرط التحقق: (CHECK)

1. تحديد الحقل الخاضع للشرط: كتابة اسم العمود ونوع بياناته الرقمية أو النصية المراد وضع معيار خاص لتقييم مدخلاتها.

3 | Grade_Score

2. استدعاء أداة الفحص: ترك مسافة فارغة، ثم كتابة العبارة المحجوزة CHECK بأحرف كبيرة للإعلان عن بدء صياغة القيد.

3 | Grade_Score INT CHECK

3. فتح نطاق الشرط: وضع قوسين هلاليين () مباشرة بعد الأداة لحصر المعادلة المنطقية بداخلهما.
4. كتابة العبارة المنطقية: صياغة الشرط داخل الأقواس باستخدام معاملات المقارنة (مثل اسم_العمود >= 18 أو اسم_العمود BETWEEN 0 AND 100).

3 | Grade_Score INT CHECK (Grade_Score BETWEEN 0 AND 100)

5. الفصل الهيكلي: وضع الفاصلة العادية ، للانتقال للعمود التالي في الجدول، أو إنهاء الأمر بالكامل بالفاصلة المنقوطة .;

```
3 | Grade_Score INT CHECK (Grade_Score BETWEEN 0 AND 100)
```

```
4 | );
```

```
CREATE TABLE IF NOT EXISTS اسم_الجدول (
    (القيمة المعامل المنطقي اسم العمود) CHECK نوع البيانات اسم العمود
);
```

```
1 CREATE TABLE IF NOT EXISTS Student_Grades (
2     Student_Id INT PRIMARY KEY,
3     Grade_Score INT CHECK (Grade_Score BETWEEN 0 AND 100)
4 );
```

منع تكرار البيانات داخل عمود معين باستخدام قيد الفريدة (UNIQUE)

تُعد هذه المهارة الأداة البرمجية القياسية لضمان عدم تطابق أو تكرار القيم في حقول معينة لا تمثل بالضرورة المفتاح الرئيسي للجدول، حيث يتمثل دورها الوظيفي في فرض شرط الفريدة على الأعمدة الحساسة (مثل البريد الإلكتروني، أو رقم الهوية الوطنية، أو رقم الهاتف). وتكمن الأهمية العملية والتعليمية لتطبيق قيد الفريدة في منع حدوث تضارب في حسابات المستخدمين أو الطلاب داخل النظام المطور؛ وتعتمد آلية عمله على بناء كشف داخلي صارم يقوم بمقارنة أي قيمة جديدة بكافة القيم السابقة المحفوظة في العمود، فإذا وجد تطابقاً، أوقف محرك القواعد عملية الحفظ فوراً وأصدر تنبيهاً يمنع تشوه سجلات الجدول.

خطوات كتابة وتطبيق قيد الفريدة: (UNIQUE)

1. تحديد الحقل الفريد: اختيار العمود المراد حمايته من تكرار البيانات (مثل حقل البريد الإلكتروني student_email).

3 | Student_Email

2. تحديد نوع البيانات: كتابة نوع البيانات النصي أو الرقمي للملائم للعمود متبوعاً بمسافة فارغة واحدة.

3 | Student_Email VARCHAR(100)

3. استدعاء أداة الفريدة: كتابة العبارة المحجوزة UNIQUE بأحرف كبيرة مباشرة بعد تحديد نوع البيانات لتعريف المحرك بشرط الحقل.

3 | Student_Email VARCHAR(100) UNIQUE

4. الجمع بين القيود (اختياري): يمكن للمطالب إلحاق القيد بعبارة NOT NULL إذا كان العمود إلزامياً وفريداً في آن واحد.

5. الفصل الهيكلي: وضع الفاصلة العادية ، في نهاية السطر للانتقال لتشديد باقي أعمدة الجدول، أو إغلاق الأمر بالفاصلة المنقوطة .

3 | Student_Email VARCHAR(100) UNIQUE
4 |);

(اسم الجدول Create Table If Not Exists

Unique نوع البيانات اسم العمود

);

```
1 CREATE TABLE IF NOT EXISTS Student_Accounts (  
2     Account_Id INT PRIMARY KEY,  
3     Student_Email VARCHAR(100) UNIQUE  
4 );
```

تفعيل خاصية الزيادة التلقائية للمفاتيح الرقمية الأساسية (AUTO_INCREMENT)

تُعد هذه المهارة الأداة البرمجية المخصصة لتوليد أرقام متسلسلة وفريدة تلقائياً للحقول التعريفية، حيث يتمثل دورها الوظيفي في إعفاء المستخدم أو المطور من كتابة المعرف الرقمي يدوياً عند إدخال سجل جديد. وتكمن الأهمية العملية والتعليمية لتطبيق خاصية الزيادة التلقائية في ضمان بناء معرفات رقمية متفردة تبدأ من الرقم (1) وتتصاعد بمقدار درجة واحدة مع كل عملية إدخال جديدة بشكل آلي ومنظم؛ وتعتمد آلية عملها على قيام محرك قواعد البيانات بإدارة عداد داخلي للجدول، بمجرد استقبال سجل جديد بدون معرف، يقوم المحرك برصد آخر رقم مسجل وزيادته تلقائياً، مما يمنع حدوث أي تكرار أو تضارب في المعرفات الأساسية للنظام.

خطوات كتابة وتفعيل خاصية الزيادة التلقائية: (AUTO_INCREMENT)

1. اختيار حقل الهوية الرقمي: تحديد العمود المخصص ليكون المعرف الأساسي للجدول وتسميته مثل معرف الطالب student_id = .

2 | Student_Id

2. إسناد النوع الرقمي: تحديد نوع البيانات الخاص بالحقل وعادة ما يكون INT لاستيعاب الأعداد الصحيحة المتسلسلة.

2 | Student_Id INT

3. تعيين القيد الرئيسي أولاً: كتابة العبارة المحجوزة PRIMARY KEY لتأمين الحقل وجعله المفتاح الأساسي للجدول.

```
2 | Student_Id INT PRIMARY KEY
```

4. استدعاء أداة الزيادة التلقائية: ترك مسافة فارغة واحدة، ثم كتابة الخاصية البرمجية AUTO_INCREMENT بأحرف كبيرة مباشرة بعد القيد الرئيسي.

```
2 | Student_Id INT PRIMARY KEY AUTO_INCREMENT
```

5. الفصل الهيكلي: وضع الفاصلة العادية ، في نهاية السطر للانتقال لتشديد باقي حقول الجدول، أو إغلاق الأمر بالفاصلة المنقوطة.

```
1 CREATE TABLE IF NOT EXISTS Students (  
2     Student_Id INT PRIMARY KEY AUTO_INCREMENT, ←  
3     Student_Name VARCHAR(100) NOT NULL  
4 ); ←
```

Create Table If Not Exists اسم_الجدول (
 اسم_العمود Int Primary Key **Auto_Increment**,
);

```
Create Table If Not Exists Students (  
    Student_Id Int Primary Key Auto_Increment,  
);
```